

Java Intermediate

Placement Training Program

Collections Framework

The Java Collections Framework provides ready-made, well-tested data structures instead of every developer building their own. `ArrayList` is a resizable array, good for indexed access and iteration; `LinkedList` is efficient for frequent insertions/removals in the middle of a list; `HashMap` stores key-value pairs with fast (average $O(1)$) lookup by key but no guaranteed order; `TreeMap` keeps keys sorted at the cost of slower operations; `HashSet` stores unique values with fast membership checks. Choosing the right collection for a task — based on whether you need ordering, uniqueness, or key-based lookup — is a core practical skill, not just memorizing the API.

Generics

Generics let a class or method work with any type while still getting compile-time type checking — `List<String>` is checked by the compiler to only ever contain `Strings`, catching type errors before the program runs rather than as a runtime `ClassCastException` the way pre-generics Java code (or an untyped language) would. Writing your own generic methods (using a type parameter like `<T>`) lets you write one algorithm — say, finding the maximum element in a list — that works correctly and safely across any comparable type, without duplicating the code for each type.

Multithreading Basics

A thread is an independent path of execution within a program; multithreading lets a program do multiple things concurrently — respond to user input while processing data in the background, for example. Java's `Thread` class and `Runnable` interface are the basic building blocks, but shared mutable state accessed from multiple threads without coordination causes race conditions — unpredictable bugs that occur only when threads happen to interleave in a particular unlucky order. The `synchronized` keyword ensures only one thread can execute a block of code at a time, which is the most basic (though not the only) tool for preventing race conditions on shared data.

File I/O

Java's I/O classes read and write data to files and other sources. The modern, recommended approach uses the `java.nio.file` package (`Files`, `Paths`) for most common operations — reading a whole file's lines, writing text to a file — which is simpler and less error-prone than the older `java.io` streams API for typical use cases. Whichever API is used, files represent an external resource that must be properly closed after use; `try-with-resources` (`try (var reader = ...) { ... }`)

automatically closes resources when the block exits, even if an exception is thrown, and should be the default pattern for any file or stream handling.

JDBC Basics

JDBC (Java Database Connectivity) is the standard API for Java programs to connect to and query relational databases like MySQL. The core flow: establish a Connection using a database URL and credentials, create a PreparedStatement with the SQL query (using placeholders for any user-supplied values, never concatenating them directly into the SQL string — this is what prevents SQL injection), execute it, and process the returned ResultSet row by row.

Understanding raw JDBC, even though most real projects use a higher-level ORM on top of it, makes it much easier to understand what an ORM is actually doing underneath and to debug when something goes wrong at the database layer.

Learn More

Oracle — The Java Tutorials: Collections

<https://docs.oracle.com/javase/tutorial/collections/>

Baeldung — Java Guides

<https://www.baeldung.com/>

GeeksforGeeks — Java Programming Language

<https://www.geeksforgeeks.org/java/>